

1.

1.1 voimme muuntaa base-10 numeron binääriksi katsomalla ensin mikä on pienin bittimäärä joka on isompi kuin numero. Sen jälkeen laskemme aloittaen vasemmalta puolelta jokaisen bitin erikseen ja katsoen onko tulos pienempi tai yhtäsuuri kuin numero, jolloin bitti on 1, muuten se on 0. Esim tehtävä a:ssa numero on 55 joten tarvitsemme 6 bittiä ($2^6 = 64$), sitten $0+32 \leq 55 = 1$. $32+16 \leq 55 = 1$. $48+8 \leq 55 = 0$. $48+4 \leq 55 = 1$. $52+2 \leq 55 = 1$. $54+1 \leq 55 = 1$. tästä saamme 110111 joka on 55 base-2 numerojärjestelmässä esitettynä.

a. 11 0111

b. 10 1010 0101

c. 100 0001

d. 10 1011 1100

1.2 2-complimentoiduiksi numeroiksi muuttaminen toimii suhteellisen samalla tavalla. Paitsi että tarvitsemme yhden bitin enemmän representoimaan onko luku negatiivinen vai positiivinen (1 tai 0). jos luku on negatiivinen, täytyy meidän ikään kuin laskea maksimista nolleen päin. Esim tehtävä d toimii näin: negatiivinen? = 1. $128-64 \geq 97 = 0$. $128-32 \geq 97 = 0$. $128-16 \geq 97 = 1$. $112-8 \geq 97 = 1$. $104-4 \geq 97 = 1$. $100-2 \geq 97 = 1$. $98-1 \geq 97 = 1$. tälläkertaa vastaus on 10011111. Ja luoja siunatkoon jos minun täytyy kirjoittaa enempää selityksiä näin ilmiselvistä laskutoimituksista tulen menettämään pinnani. :(

a. 0011 1011

b. 1111 1111

c. 1000 0000

d. 1001 1111

1.3

a. 0x1E5 takes up 9 bits (0001 1110 0101) so it can't be converted into 8 bits.

b. 1111 0001

c. 10 1110 1000

d. 0100 1000 1011

e. 0000 0000 1101 1001

1.4

a. 1111 1001 1101

b. 1110 1011 1001 0001 takes up at least 14 bits due to the '0' at the 13th bit, and so can't be converted into 12 bits.

c. 1111 1111 1010 1100

d. 11 0001 0011

1.5

See 'bitwise.py' attachment